

Convolution Neural Network

October 29, 2024

```
[2]: # importing the necessary libraries

import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

```
[3]: # load the train data
train = pd.read_csv("train.csv")
train.head()
```

```
[3]:
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	\
0	1	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	1	0	0	0	0	0	0	0	0	
3	4	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	

	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	\
0	0	...	0	0	0	0	0	0	
1	0	...	0	0	0	0	0	0	
2	0	...	0	0	0	0	0	0	
3	0	...	0	0	0	0	0	0	
4	0	...	0	0	0	0	0	0	

	pixel780	pixel781	pixel782	pixel783
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0

[5 rows x 785 columns]

```
[4]: train.shape
```

```
[4]: (42000, 785)
```

```
[5]: train.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 42000 entries, 0 to 41999  
Columns: 785 entries, label to pixel783  
dtypes: int64(785)  
memory usage: 251.5 MB
```

```
[6]: test = pd.read_csv('test.csv')  
test.head()
```

```
[6]:   pixel0  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  pixel8  \  
0         0         0         0         0         0         0         0         0         0  
1         0         0         0         0         0         0         0         0         0  
2         0         0         0         0         0         0         0         0         0  
3         0         0         0         0         0         0         0         0         0  
4         0         0         0         0         0         0         0         0         0  
  
   pixel9  ...  pixel774  pixel775  pixel776  pixel777  pixel778  pixel779  \  
0         0  ...         0         0         0         0         0         0  
1         0  ...         0         0         0         0         0         0  
2         0  ...         0         0         0         0         0         0  
3         0  ...         0         0         0         0         0         0  
4         0  ...         0         0         0         0         0         0  
  
   pixel780  pixel781  pixel782  pixel783  
0           0           0           0           0  
1           0           0           0           0  
2           0           0           0           0  
3           0           0           0           0  
4           0           0           0           0
```

```
[5 rows x 784 columns]
```

```
[7]: test.shape
```

```
[7]: (28000, 784)
```

```
[8]: test.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 28000 entries, 0 to 27999  
Columns: 784 entries, pixel0 to pixel783  
dtypes: int64(784)  
memory usage: 167.5 MB
```

```
[9]: # put the labels columns as y_train

y_train = train['label']
X_train = train.drop(labels = ['label'], axis = 1)
```

```
[10]: y_train.head()
```

```
[10]: 0    1
      1    0
      2    1
      3    4
      4    0
      Name: label, dtype: int64
```

```
[11]: X_train.head()
```

```
[11]:  pixel0  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  pixel8  \
0         0         0         0         0         0         0         0         0         0
1         0         0         0         0         0         0         0         0         0
2         0         0         0         0         0         0         0         0         0
3         0         0         0         0         0         0         0         0         0
4         0         0         0         0         0         0         0         0         0

      pixel9  ...  pixel774  pixel775  pixel776  pixel777  pixel778  pixel779  \
0         0  ...         0         0         0         0         0         0
1         0  ...         0         0         0         0         0         0
2         0  ...         0         0         0         0         0         0
3         0  ...         0         0         0         0         0         0
4         0  ...         0         0         0         0         0         0

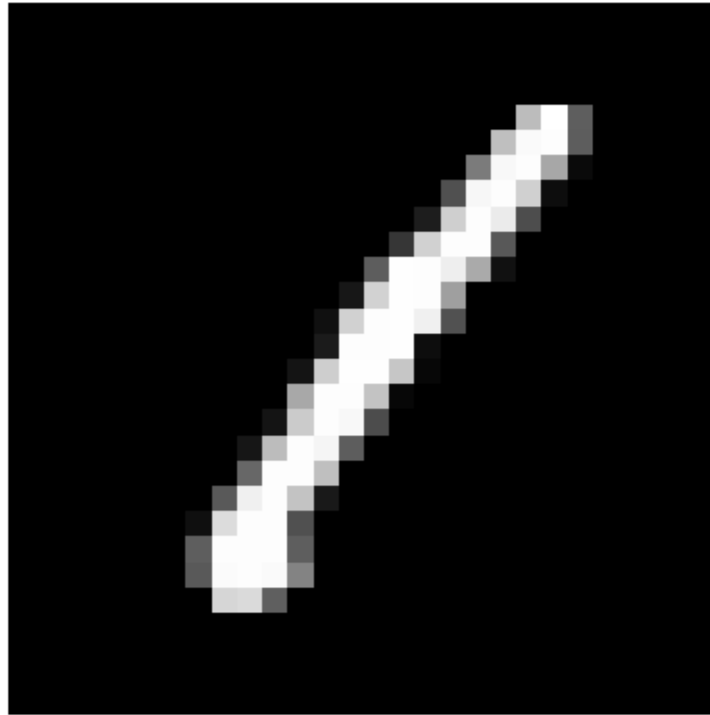
      pixel780  pixel781  pixel782  pixel783
0         0         0         0         0
1         0         0         0         0
2         0         0         0         0
3         0         0         0         0
4         0         0         0         0
```

[5 rows x 784 columns]

```
[12]: #plot some samples in the train

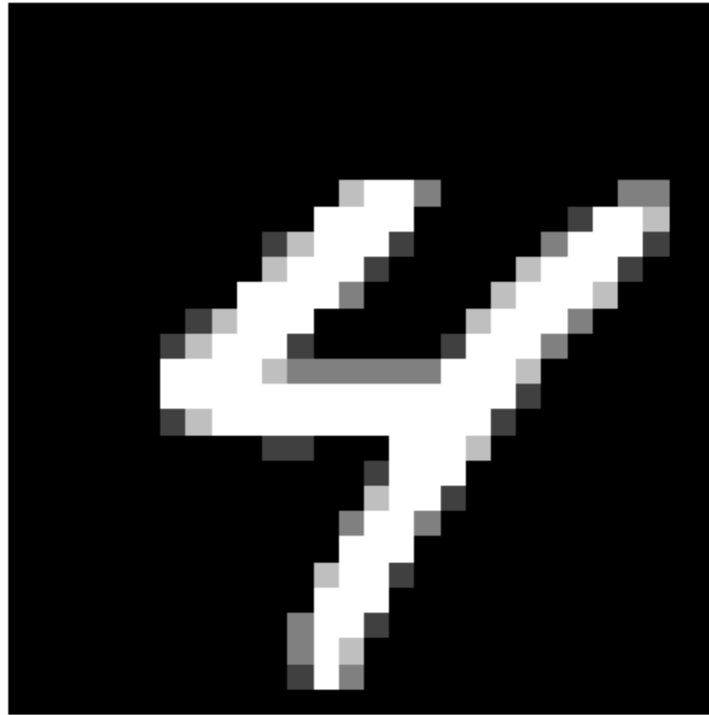
img = X_train.iloc[0].values
img = img.reshape((28,28))
plt.imshow(img, cmap = 'gray')
plt.title('train.iloc[0,0]')
plt.axis('off')
plt.show()
```

train.iloc[0,0]



```
[13]: img = X_train.iloc[250].values
      img = img.reshape((28,28))
      plt.imshow(img, cmap = 'gray')
      plt.title('train.iloc[250,500]')
      plt.axis('off')
      plt.show()
```

train.iloc[250,500]



```
[14]: y_train.value_counts()
```

```
[14]: 1    4684
      7    4401
      3    4351
      9    4188
      2    4177
      6    4137
      0    4132
      4    4072
      8    4063
      5    3795
      Name: label, dtype: int64
```

```
[15]: # plot the number of digits in the label columns

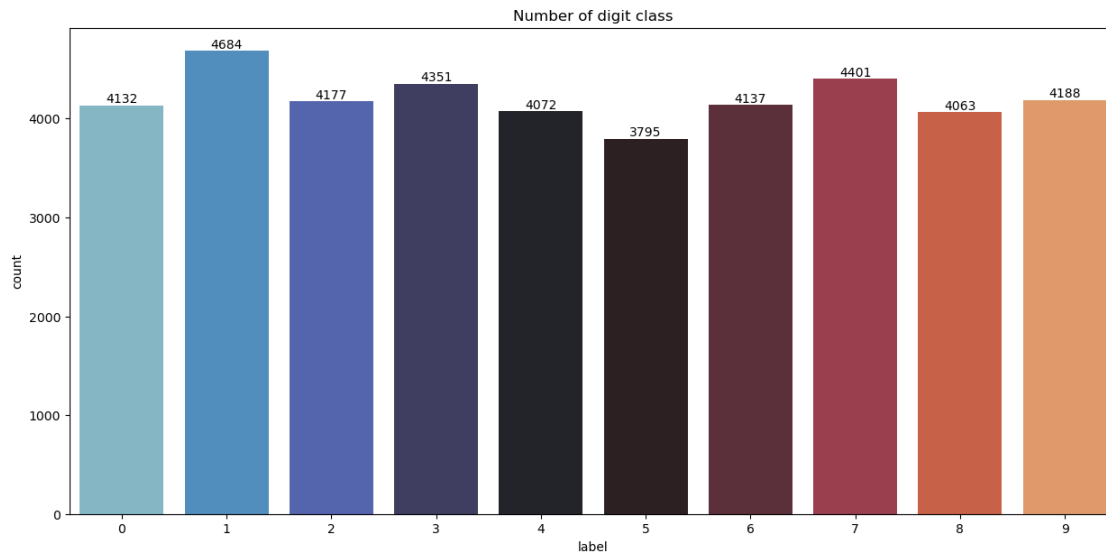
plt.figure(figsize = (15,7))
g = sns.countplot(x = y_train, palette = 'icefire')

# add labels to the bar
for patch in g.patches:
```

```

    height = patch.get_height()
    g.text(patch.get_x() + patch.get_width()/2, height + 0.5, int(height), ha = 'center', va = 'bottom')
plt.title("Number of digit class")
plt.show()

```



```

[16]: # check for missing values in any rows

missing_values = train.isnull().any(axis=1)
count_missing_values = missing_values.sum()
count_missing_values

```

[16]: 0

```

[17]: missing_values = test.isnull().any(axis=1)
count_missing_values = missing_values.sum()
count_missing_values

```

[17]: 0

```

[18]: # Both train and test does not have any missing values
# Normalize the image data
X_train = X_train/255.0
test = test/255.0

X_train.shape, test.shape

```

[18]: ((42000, 784), (28000, 784))

```
[19]: # we need to reshape the images and perform a grayscale to reduce the effect of  
      ↪ illumination and perform normalization for all data
```

```
X_train = X_train.values.reshape(-1,28,28,1)  
test = test.values.reshape(-1,28,28,1)  
X_train.shape, test.shape
```

```
[19]: ((42000, 28, 28, 1), (28000, 28, 28, 1))
```

```
[20]: #pip install tensorflow
```

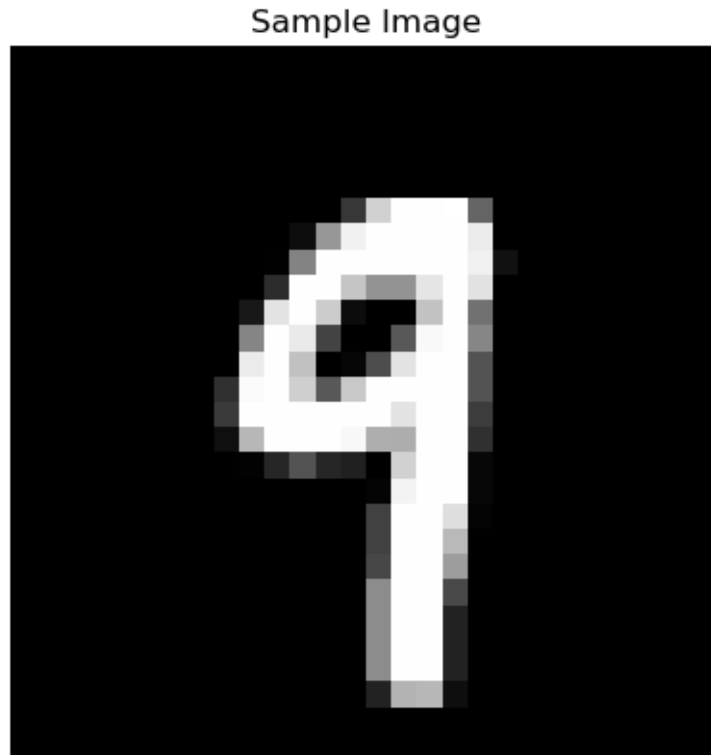
```
[21]: # We will train_test_split  
  
# Convert the labels to one-hot encoding  
from sklearn.model_selection import train_test_split  
from tensorflow.keras.utils import to_categorical  
num_classes = 10  
y_train = to_categorical(y_train,num_classes = num_classes)  
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size =  
      ↪ 0.1, random_state=2)
```

```
2024-10-29 03:27:06.590722: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]  
Could not find cuda drivers on your machine, GPU will not be used.  
2024-10-29 03:27:06.596146: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]  
Could not find cuda drivers on your machine, GPU will not be used.  
2024-10-29 03:27:06.613622: E  
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:485] Unable to register  
cuFFT factory: Attempting to register factory for plugin cuFFT when one has  
already been registered  
2024-10-29 03:27:06.640894: E  
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:8454] Unable to register  
cuDNN factory: Attempting to register factory for plugin cuDNN when one has  
already been registered  
2024-10-29 03:27:06.649280: E  
external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1452] Unable to  
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when  
one has already been registered  
2024-10-29 03:27:06.669800: I tensorflow/core/platform/cpu_feature_guard.cc:210]  
This TensorFlow binary is optimized to use available CPU instructions in  
performance-critical operations.  
To enable the following instructions: AVX2 AVX512F FMA, in other operations,  
rebuild TensorFlow with the appropriate compiler flags.  
2024-10-29 03:27:08.986135: W  
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not  
find TensorRT
```

```
[22]: X_train.shape, X_val.shape, y_train.shape, y_val.shape
```

```
[22]: ((37800, 28, 28, 1), (4200, 28, 28, 1), (37800, 10), (4200, 10))
```

```
[23]: plt.imshow(X_train[2][:,:,0], cmap = 'gray')
plt.title('Sample Image')
plt.axis('off')
plt.show()
```



```
[24]: from sklearn.metrics import confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dropout, Dense, Flatten, Conv2D, MaxPool2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
[33]: # Build the CNN model

model = Sequential()
model.add(Conv2D(filters = 8, kernel_size=(5,5), padding = 'Same', activation = 'relu', input_shape = (28,28,1)))
model.add(MaxPool2D(pool_size=(2,2)))
model.add(Dropout(0.25))
```

```
model.add(Conv2D(filters = 16, kernel_size=(3,3), padding = 'Same', activation_↵
↵= 'relu'))
model.add(MaxPool2D(pool_size=(2,2), strides = (2,2)))
model.add(Dropout(0.25))

model.add(Flatten())
model.add(Dense(256, activation = 'relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation = 'softmax'))
```

```
[ ]: # Define the optimizer , To be continued
```